Full Sail University BSIMM Assessment

Table of Contents

Introduction *2*

Governance *3*

Strategy & Metrics **3**

SM1.1 - Publish Process and Evolve as Necessary **3**

SM2.7 - Create or Grow a Security Champions Program **3**

Compliance & Policy *4*

CP1.1- Identify Privacy Obligations **4**

CP1.2- Create Policy **4**

CP1.3 - Build a PII Inventory **5**

CP2.5 - Ensure Executive Awareness of Compliance and Privacy Obligations **5**

Training *5*

T1.1 - Conduct Software Security Awareness Training **5**

T1.7 - Deliver On-Demand Individual Training **6**

T2.8 - Reward Progression Through Curriculum **6**

T2.9 -Provide Expertise via Open Collaboration Channels **6**

Intelligence *7*

Attack Models **7**

AM1.5 - Gather and Use Attack Intelligence 7

AM2.2 - Build Attack Patterns and Abuse Cases 7

AM2.5 - Create Technology-Specific Attack Patterns 7

Security Features and Design *8*

SFD1.1 - Integrate and Deliver Security Features **8**

SFD1.2 - Application Architecture Teams Engage with Security **8**

Standards and Requirements *9*

SR1.1 - Create Security Standards **9**

SR1.3 - Translate Compliance Constraints into Requirements **9**

SR2.2 - Create SLA Boilerplate **9**

SR2.5 - Control Open-Source Risk **10**

SSDLC Touchpoints *10*

Architecture Analysis **10**
AA1.1 - Perform Security Feature Reviews 10
AA1.2 - Perform Design Reviews for High-Risk Applications 10
AA2.2 - Engineering Teams Lead Architecture Analysis 11
AA2.4 - Drive Analysis Results into Architecture Patterns 11
Code Review ***11***
CR1.2 - Use Automated Code Review Tools **11**
CR1.4 - Make Code Review Mandatory **12**
CR2.8 - Enforce Secure Coding Standards **12**
Security Testing ***12***
ST1.1 - Perform Edge and Boundary Testing During QA **12**
ST1.3 - Integrate Security Tools into QA **13**
Deployment ***13***
Penetration Testing **13**
PT1.1 - Use External Penetration Testers 13
PT1.3 - Use Penetration Testing Tools Internally 13
Software Environment ***14***
SE1.2 - Ensure Host and Network Security Basics Are in Place **14**
Configuration Management & Vulnerability Management ***14***
CMVM1.1 - Create or Interface with Incident Response **14**
CMVM2.1 - Have Emergency Response Capability **15**
CMVM3.4 - Enhance the SSDLC Based on Operational Findings **15**
Conclusion ***15***

Introduction

This entirety of this document has been established upon the previous security incident involving the Full Sail Online (FSO) application. Full Sail University assessed its software security practices using the Building Security in Maturity Model (BSIMM) framework. The purpose of this assessment is to identify gaps in the university's current software development practices and provide recommendations that align with BSIMM requirements. The assessment entails

Governance, Intelligence, SSDLC Touchpoints, and Deployment activities to improve Full Sail's overall software security posture.

Governance

Strategy & Metrics

SM1.1 - Publish Process and Evolve as Necessary

Current State:

Full Sail's development teams primarily focus on delivering requested features. While there is an informal process to prevent application failures during deployment, no documented software security process exists.

Gap:

BSIMM requires organizations to establish and publish a software security process that can be improved over time. Full Sail lacks formal software security governance.

Recommendation:

Develop and publish a Software Security Development Lifecycle (SSDLC) that incorporates security requirements, secure coding practices, code reviews, testing procedures, and deployment standards. Review and update the SSDLC regularly.

SM2.7 - Create or Grow a Security Champions Program

Current State:

One developer is interested in secure coding practices but lacks support, resources, and executive sponsorship.

Gap:

No formal security champion program exists.

Recommendation:

Establish a Security Champion program that designates security-focused developers within each development team. Provide training and leadership support to encourage secure development practices throughout the organization.

Compliance & Policy

CP1.1- Identify Privacy Obligations

Current State:

Full Sail complies with FERPA and GLBA requirements through the efforts of a compliance specialist.

Gap:

The organization has not fully identified where privacy-related data exists throughout its applications.

Recommendation:

Conduct a privacy impact assessment and identify all applications that collect, process, store, or transmit protected information.

CP1.2- Create Policy

Current State:

No formal software security policy exists.

Gap:

Faculty and developers lack documented guidance regarding security responsibilities.

Recommendation:

Develop comprehensive software security policies covering secure development, access controls, data protection, vulnerability management, and incident response.

CP1.3 - Build a PII Inventory

Current State:

Limited visibility exists regarding how personally identifiable information is handled.

Gap:

The organization lacks a documented inventory of PII.

Recommendation:

Create and maintain a centralized inventory of all PII processed by university systems and applications.

CP2.5 - Ensure Executive Awareness of Compliance and Privacy Obligations

Current State:

Executives receive compliance training related to FERPA and GLBA.

Gap:

Training does not adequately address the consequences of non-compliance.

Recommendation:

Expand executive training to include legal penalties, breach consequences, financial impacts, and reputational risks associated with compliance failures.

Training

T1.1 - Conduct Software Security Awareness Training

Current State:

Faculty receive general security awareness training.

Gap:

Developers do not receive software security-specific training.

Recommendation:

Implement secure coding awareness training for all software developers.

T1.7 - Deliver On-Demand Individual Training

Current State:

No software security learning resources are available to developers.

Gap:

Developers cannot access security training when needed.

Recommendation:

Provide access to online secure coding courses, videos, and learning portals.

T2.8 - Reward Progression Through Curriculum

Current State:

No developer security training program exists.

Gap:

There are no incentives to encourage security skill development.

Recommendation:

Create a developer security learning path and reward employees who complete training milestones.

T2.9 -Provide Expertise via Open Collaboration Channels

Current State:

Communication between developers and security professionals is limited.

Gap:

Developers lack easy access to security expertise.

Recommendation:

Create dedicated collaboration channels such as Microsoft Teams or Slack for software security discussions.

Intelligence

Attack Models

AM1.5 - Gather and Use Attack Intelligence

Current State:

Full Sail subscribes to a commercial threat intelligence service.

Gap:

Threat intelligence is not integrated into software development activities.

Recommendation:

Use threat intelligence reports during risk assessments, design reviews, and threat modeling exercises.

AM2.2 - Build Attack Patterns and Abuse Cases

Current State:

The security team maintains a list of the top five threats facing the university.

Gap:

Application-specific attack scenarios have not been developed.

Recommendation:

Create attack patterns and abuse cases for critical university applications.

AM2.5 - Create Technology-Specific Attack Patterns

Current State:

Threats are evaluated at a high level.

Gap:

No technology-specific attack patterns exist.

Recommendation:

Develop attack models tailored to web applications, APIs, cloud systems, databases, and authentication services.

Security Features and Design

SFD1.1 - Integrate and Deliver Security Features

Current State:

The security team maintains infrastructure security controls, but communication with developers is inconsistent.

Gap:

Security controls are not integrated into application development requirements.

Recommendation:

Document required security features and incorporate them into application design and development standards.

SFD1.2 - Application Architecture Teams Engage with Security

Current State:

There is no formal collaboration process between development and security teams.

Gap:

Security guidance is not consistently incorporated into application architecture decisions.

Recommendation:

Establish recurring security architecture review meetings throughout the development lifecycle.

Standards and Requirements

SR1.1 - Create Security Standards

Current State:

Few documented security standards exist.

Gap:

Developers lack formal guidance on secure development practices.

Recommendation:

Create standards for secure coding, authentication, encryption, logging, and vulnerability management.

SR1.3 - Translate Compliance Constraints into Requirements

Current State:

Compliance requirements are reviewed annually.

Gap:

Compliance obligations are not consistently translated into technical requirements.

Recommendation:

Convert FERPA and GLBA requirements into actionable software security requirements.

SR2.2 - Create SLA Boilerplate

Current State:

Vendor SLAs do not contain software security requirements.

Gap:

Third-party vendors are not contractually required to meet security expectations.

Recommendation:

Update vendor contracts to include security testing, vulnerability management, and breach notification requirements.

SR2.5 - Control Open-Source Risk

Current State:

No formal process exists to manage open-source software risks.

Gap:

The organization lacks visibility into open-source dependencies and vulnerabilities.

Recommendation:

Implement Software Composition Analysis (SCA) tools and maintain a Software Bill of Materials (SBOM).

SSDLC Touchpoints

Architecture Analysis

AA1.1 - Perform Security Feature Reviews

Current State:

No security design reviews are performed.

Gap:

Applications are not evaluated for security requirements during design.

Recommendation:

Perform security feature reviews during architecture planning.

AA1.2 - Perform Design Reviews for High-Risk Applications

Current State:

No formal design review process exists.

Gap:

High-risk applications may contain architectural weaknesses.

Recommendation:

Conduct security-focused design reviews for critical systems.

AA2.2 - Engineering Teams Lead Architecture Analysis

Current State:

Developers are interested in security but lack formal processes.

Gap:

Architecture analysis is not integrated into development workflows.

Recommendation:

Train development teams to conduct security architecture reviews.

AA2.4 - Drive Analysis Results into Architecture Patterns

Current State:

Security lessons learned are not documented.

Gap:

Knowledge is not reused across projects.

Recommendation:

Develop secure architecture patterns and apply them across future projects.

Code Review

CR1.2 - Use Automated Code Review Tools

Current State:

Some developers independently use SonarQube.

Gap:

Automated code analysis is not standardized.

Recommendation:

Implement a mandatory enterprise-wide static application security testing solution.

CR1.4 - Make Code Review Mandatory

Current State:

Code review requirements vary by developer.

Gap:

Security vulnerabilities may enter production without review.

Recommendation:

Require peer code reviews before all production deployments.

CR2.8 - Enforce Secure Coding Standards

Current State:

No formal secure coding standards exist.

Gap:

Developers lack consistent security expectations.

Recommendation:

Publish secure coding standards and integrate them into code review processes.

Security Testing

ST1.1 - Perform Edge and Boundary Testing During QA

Current State:

QA activities focus on functionality and performance.

Gap:

Security testing is not included.

Recommendation:

Incorporate boundary testing and security validation into QA procedures.

ST1.3 - Integrate Security Tools into QA

Current State:

No security testing tools are integrated into QA.

Gap:

Security defects may remain undetected.

Recommendation:

Implement automated dynamic application security testing tools within the QA process.

Deployment

Penetration Testing

PT1.1 - Use External Penetration Testers

Current State:

Annual third-party penetration testing is performed.

Gap:

Testing occurs only once per year.

Recommendation:

Increase testing frequency for critical applications and major changes.

PT1.3 - Use Penetration Testing Tools Internally

Current State:

No internal penetration testing capabilities exist.

Gap:

Security weaknesses may go undetected between annual assessments.

Recommendation:

Train internal personnel and implement tools such as Nessus and Burp Suite for ongoing testing.

Software Environment

SE1.2 - Ensure Host and Network Security Basics Are in Place

Current State:

Security baselines exist for production infrastructure but not development environments.

Gap:

Development environments may introduce security weaknesses.

Recommendation:

Apply hardening standards and security baselines to development systems and tools.

Configuration Management & Vulnerability Management

CMVM1.1 - Create or Interface with Incident Response

Current State:

An Incident Response Plan exists and includes backup restoration procedures.

Gap:

Developer responsibilities during incidents are not clearly defined.

Recommendation:

Integrate software development teams into incident response planning and execution.

CMVM2.1 - Have Emergency Response Capability

Current State:

No emergency software patching process exists.

Gap:

Critical vulnerabilities cannot be rapidly remediated.

Recommendation:

Develop and test an emergency software change process for security patches.

CMVM3.4 - Enhance the SSDLC Based on Operational Findings

Current State:

Operational security findings are not systematically incorporated into development activities.

Gap:

Recurring vulnerabilities may continue to occur.

Recommendation:

Create a feedback process that uses incident data and vulnerability findings to improve training, standards, architecture reviews, and testing procedures.

Conclusion

The BSIMM assessment demonstrates that Full Sail University has established a foundation of compliance and security awareness activities. However, significant gaps remain in secure software development governance, training, architecture analysis, code review, security testing, and vulnerability management. By implementing the recommendations outlined in this assessment, Full Sail will perceptibly establish a mature Software Security Initiative that better protects university systems, student information, and institutional resources while reducing the likelihood and impact of future security incidents.